

A Multiple-Neighborhood-Based Parallel Composite Local Search Algorithm for Timetable Problem

YAN He(颜鹤), YU Song-nian(郁松年)

School of Computer Engineering and Science, Shanghai University, Shanghai 200072, P. R. China

Abstract This paper presents a parallel composite local search algorithm based on multiple search neighborhoods to solve a special kind of timetable problem. The new algorithm can also effectively solve those problems that can be solved by general local search algorithms. Experimental results show that the new algorithm can generate better solutions than general local search algorithms.

Key words multiple neighborhoods, parallel, composite local search algorithm, timetable problem.

1 Introduction

It is well known that timetabling problems are very difficult to solve, especially when dealing with large instances. The timetable problem is known in general to be NP-hard^[1]. Since there are many strict constraints to be satisfied, it is difficult to generate a satisfactory solution within a short time.

The timetable problem can simply be defined as follows^[2]. Allocate a number of events to a finite number of time slots (or periods) so that the necessary constraints are satisfied. A large number of variations of timetable problem have been proposed in the literature. In this paper we solve a special kind of timetable problem—coursetable problem. A coursetable problem consists in scheduling lectures for a set of courses in a university, taught by available teachers in a given period composing a number of weeks, and in available classrooms. Although the constraints of timetable problem vary from case to case, one can classify all constraints into two categories: hard constraints and soft constraints. Hard constraints must be satisfied strictly because any timetable that violates just one will become useless. A timetable that violates some soft constraints can still be usable although it may cause some inconvenience to the users. It is often

very difficult to satisfy all the soft constraints in a real-life timetable problem. Some concrete definition of hard constraints and soft constraints in a coursetable problem will be given in the second part of the paper.

In order to ascertain the quality of a timetable, the penalty function is applied to it. The penalty function of a timetable usually calculates the weighted sum of times it violates hard constraints and the times it violates soft constraints. The weight for hard constraints is much larger than that for soft constraints. Now the timetable problem is converted to an equivalent problem of minimizing the value of penalty function over all possible cases.

In the last two decades, many methods have been proposed and applied to the timetable problem, some of which are listed below^[3].

- For the first time Breslaw used integer linear programming to solve timetable problems in 1976.
- In 1976 Dyer and Mulvey proposed a way that used a network mode as a core of the timetable algorithm.
- In 1985 de Werra showed how to transform a timetable problem into a graph coloring problem.
- In 1987 Jaffar and Lassez presented a new idea to use constraints logic programming method to solve timetable problems.
- In 1991 the tabu search was first applied to timetable problems by Hertz.
- Meisels, Gudes and Kuflik proposed a solution technique based on expert systems to solve timetable problems in 1991.
- In 1992 Ling first used genetic algorithm to solve timetable problems.

Up to now, almost all new techniques dealing with

Received Jun. 3, 2003; Revised Nov. 5, 2003

Project supported by the Science Foundation of Shanghai Municipal Commission of Science and Technology (Grant No. 00JC14052) and the Science Foundation of Shanghai Municipal Commission of Education

YAN He, M. En. Candidate, E-mail: yanhe790926@sina.com;

YU Song-nian, Ph. D., Prof., E-mail: yywww@sh163.com

timetable problems are based on these methods. we will concentrate more on the tabu search algorithm, simulated annealing algorithm and genetic algorithm, and show their advantages and disadvantages. An algorithm is proposed in this paper based on these heuristic algorithms and it overcomes some disadvantages of the previous techniques.

The paper is organized as follows. In Section 2, some definitions for coursetable problems are given. In Section 3 the proposed algorithm is described, and the application of the algorithm to coursetable problems is discussed. Section 4 gives experimental results and discussion. The final section draws conclusions and gives directions of the future work.

2 Some Definitions about the Coursetable Problem

There are m courses $C_1, C_2, \dots, C_m$, n teachers $J_1, J_2, \dots, J_n$, p periods $1, 2, \dots, p$, and k classrooms $R_1, R_2, \dots, R_k$. Each course C_i consists of L_i lectures to be scheduled in different periods, and is taken by S_i students. Each classroom R_i has a capacity of Y_i seats.

Generation of a coursetable can be divided into two parts^[4]. The first part is to build a relation between teachers and courses, taking into account certain priorities. A teacher may teach several courses, and a course can be taught by several teachers. The second part is to allocate a feasible period and a feasible classroom to each lecture of every course. For simplicity, we suppose that the first part has already been accomplished. In other words, the relation between teachers and courses has been determined. Now the coursetable problem has become one of allocating a feasible period and an available classroom to each lecture of every course under some constraints.

The coursetable can be represented by a matrix $T = (t_{ij})_{m \times p}$ where $t_{ij} = h$ ($h \in \{1, 2, \dots, n\}$) means that the course C_i has a lecture in classroom R_h in period j , and $t_{ij} = 0$ means that the course C_i has no lecture in period j . For example, $t_{23} = 4$ means that course C_2 has a lecture in classroom R_4 in period 3. If t_{12} is zero, course C_1 has no lecture in period 2. Some other definitions will be given as follows^[5].

Definition 1 B is a course-period matrix that does not include information about room. The relation between matrix B and matrix T is as follows:

$$B = (b_{ij})_{m \times p}, b_{ij} = \begin{cases} 0, & \text{if } t_{ij} = 0, \\ 1, & \text{if } t_{ij} > 0. \end{cases}$$

Definition 2 Z is a course confliction matrix. Two courses are said to be in conflict when there are students enrolled in both courses.

$$Z = (z_{ij})_{m \times m}, z_{ij} = \begin{cases} 0, & \text{if no conflict exists between } c_i \text{ and } c_j, \\ 1, & \text{otherwise.} \end{cases}$$

Definition 3 H is a teacher availability matrix.

$$H = (h_{ij})_{n \times p}, h_{ij} = \begin{cases} 0, & \text{if teacher } J_i \text{ is not available in period } j, \\ 1, & \text{otherwise.} \end{cases}$$

Definition 4 A is a course availability matrix. According to the relation between teachers and courses, a teacher availability matrix H can be converted into a course availability matrix.

$$A = (a_{ij})_{m \times p}, a_{ij} = \begin{cases} 0, & \text{if course } C_i \text{ is not available in period } j, \\ 1, & \text{otherwise.} \end{cases}$$

Definition 5 Every course has a min-time-span value. The min-time-span of a course is the least interval between the first lecture and the last lecture of the course. D_i denotes the min-time-span of course C_i .

To generate a satisfactory timetable, we must consider all the hard constraints and try to satisfy all the soft constraints if possible. The hard constraints and soft constraints of the coursetable problem are defined as follows^[6,7]:

(1) Hard constraints

(i) The number of lectures of each course C_i must be exactly equal to L_i .

$$\forall i \in \{1, 2, \dots, m\}, \sum_{j=1}^p b_{ij} = L_i.$$

(ii) Two different lectures can not take place in the same classroom and in the same period.

$$\forall j \in \{1, 2, \dots, p\}, \forall i_1, i_2 \in \{1, 2, \dots, m\}, \text{ if } i_1 \neq i_2, \text{ then } t_{i_1 j} \neq t_{i_2 j}.$$

(iii) Two conflicting courses can not take place in the same period.

$$\forall j \in \{1, 2, \dots, p\}, \forall i_1, i_2 \in \{1, 2, \dots, m\}, \text{ if } i_1 \neq i_2 \text{ and } z_{i_1 i_2} = 1, \text{ then } t_{i_1 j} \times t_{i_2 j} = 0.$$

(iiii) Every course might take place in these periods in which the course is available.

$$\forall i \in \{1, 2, \dots, m\}, \forall j \in \{1, 2, \dots, p\}, \text{ if } t_{ij} > 0, \text{ then } a_{ij} = 1.$$

(2) Soft constraints

(i) The number of students that attend a course must be less than or equal to the number of seats of all classrooms that host its lectures.

$\forall i \in \{1, 2, \dots, m\}, \forall j \in \{1, 2, \dots, p\}, \text{ if } t_{ij} > 0, \text{ then } Y_{t_{ij}} \geq S_i.$

(ii) The interval between the first and the last lectures of every course must be more than min-time-span of the course.

Symbols used to represent hard and soft constraints are $H1, H2, H3, H4$ and $S1, S2$ respectively. We define a function $B(x)$ to calculate the times that a coursetable breaks the constraint x . So $B(H1), B(H2), B(H3)$, and $B(H4)$ calculate the times a coursetable breaks $H1, H2, H3, H4$ respectively. $F(S1)$ and $F(S2)$ calculates the times a coursetable breaks $S1$ and $S2$ respectively. In this way, the following penalty function can show the quality of a coursetable T .

$$\text{Penalty}(T) = [B(H1) + B(H2) + B(H3) + B(H4)] \times 1000 + [B(S1) + B(S2)].$$

Notice that the weight for hard constraints is 1000 times greater than that for soft constraints. This means that hard constraints were taken into consideration much more strictly than soft constraints. The less the value of $\text{Penalty}(T)$, the better the quality of the course coursetable T . If there is a coursetable with zero value of its penalty function, it must be optimal.

3 Multiple-Neighborhood-Based Parallel Composite Local Search Algorithm for Coursetable Problem

The multiple neighborhood based parallel composite local search algorithm is based on some general local search algorithms and overcomes some disadvantages of general local search algorithms. In the first part of this section we briefly introduce some general local search algorithms. In the second part the multiple-neighborhood-based parallel composite local search algorithm is presented. Finally the multiple-neighborhood-based parallel composite local search algorithm is applied to solve coursetable problems.

3.1 General local search algorithm

Local search algorithm is a family of general-purpose algorithms for search and optimization problems that are based on several variants of a simple iterative idea. Given a solution of a problem instance they try

to fix in order to obtain a better solution, and they iteratively continue until no improvement exists anymore.

The most popular local search algorithm are hill climbing, simulated annealing, and tabu search, based on three different philosophies: simple, probabilistic and memory-based, respectively^[8].

• Hill climbing

The algorithm is based on the simple idea of carrying out a move only when it improves the cost function or leaves it unchanged. The main problem of this algorithm is the fact that it can easily get stuck in so-called local minima.

• Simulated annealing (SA)

SA is a probabilistic local search algorithm whose name comes from the fact that it simulates the cooling of a collection of hot vibrating atoms. After a random initial solution is created, the algorithm generates a move m at each iteration, and it performs m according to the following rule. The move is certainly accepted if it improves the cost value of the current state, or accepted according to a time-decreasing probability. In the latter case, if the acceptance test is negative then no move is performed, and the current state is left unchanged.

• Tabu search (TS)

TS is a method that keeps track of features of previous solutions in order to inhibit the search to visit them again. The basic mechanism of TS is quite simple. At each iteration a subset of the neighborhood of the current solution is explored, and the neighbor that gives the minimum value of the cost function becomes a new current solution independent of the fact that its value is better or worse than the value of the current solution. The subset to be explored at each step is determined by a so-called tabu list, which is a short-term memory of moves that are forbidden to execute. These moves are not taken into account in the exploration of the neighborhood, because they can lead to already visited solutions.

3.2 Multiple-neighborhood-based parallel composite local search algorithm

All the local search algorithms mentioned in Section 3.1 are based on a single search neighborhood. One of the main issues of these local search algorithms is to avoid local minimum solutions. Although certain local search algorithms employ some specific mechanism such as SA and TS for this purpose, experimental results indicate that they are not very effective.

Intuitively, it would be effective to avoid local minima by using multiple search neighborhoods. There are two ways to combine multiple search neighborhoods with an algorithm. One way is to combine a composite algorithm with multiple search neighborhoods. These algorithms still have a single search neighborhood, while the composite algorithm already has multiple search neighborhoods no matter what kinds of algorithms form it. We have already done some experiments in this way, and the result is pretty good (See Section 4). Another possible method is to embed a single local search algorithm such as SA and TS into multiple search neighborhoods. This will be done in the future.

In the experiments several composite local search algorithms were performed with the same random initial solution simultaneously in a clustering system^[9]. The reasons for doing so are listed as follows:

- Several different near-optimal solutions can be obtained in one execution. Different near-optimal solutions are needed depending on different criteria.
- When a random initial solution is obtained, one does not know exactly which kind of composite local search algorithms is the most effective. So several composite local search algorithms are tested for the same initial solution at the same time.
- If experimental data about the relation between initial solution and composite local search algorithm are obtained, one can know which kind of composite local search algorithms is the most effective for which kind of the initial solution. It is useful to develop a theory for the composite algorithm, because develop-

ment of the theory takes more time than the development of applications of the algorithm.

Suppose two search neighborhoods, named $N1$ and $N2$ for a problem, and two general local search algorithms, named $L1$ and $L2$, combined with the two search neighborhoods are defined^[10]. For example, $L2(N1)$ denotes algorithm $L2$ combined with search neighborhood $N1$. There are four kinds of composite local search algorithms: $L1(N1) + L2(N2)$, $L1(N2) + L2(N1)$, $L2(N1) + L1(N2)$, and $L2(N2) + L1(N1)$. Fig. 1 shows the framework of multiple-neighborhood-based parallel composite local search algorithms.

3.3 Multiple-neighborhood-based parallel composite local search algorithm for the coursetable problem

The detailed definitions of coursetable problem have been given in Section 2. Here some explanation about the "state" is in order. As a term in local search algorithms, state represents different things depending on the specific problems. In the coursetable problem the state represents a coursetable and it can be viewed as a matrix like the coursetable.

Given an instance p of a problem P , we associate a search space S with it^[11,12]. Each element $s \in S$ corresponds to a potential solution of p , and is called a state of P . A local search algorithm relies on a move M which assigns each $s \in S$ to its neighborhood $N(s) \in S$. Each state $s' \in N(s)$ is called a neighbor of s . The definition of move is the key of a local search algorithm.

Now we define two kinds of move for coursetable

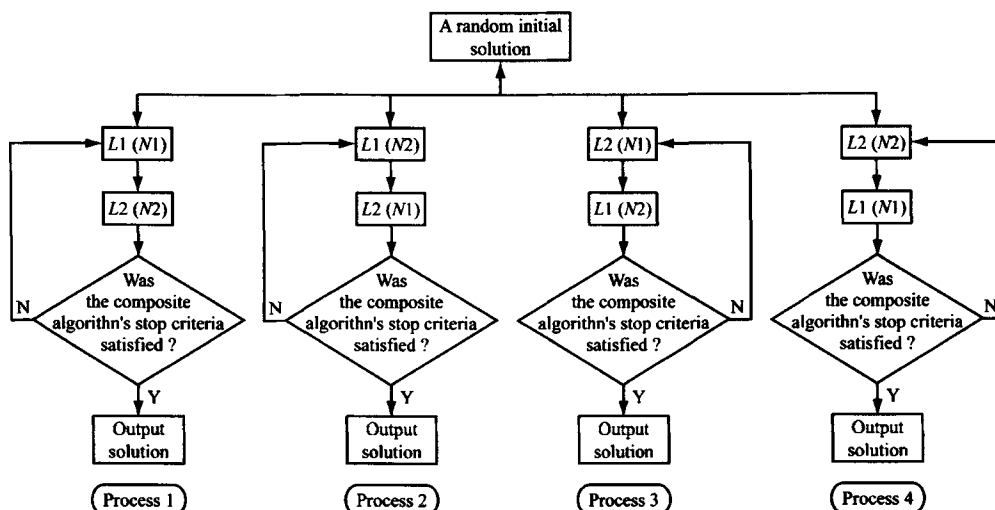


Fig. 1 Framework of multiple-neighborhood-based parallel composite local search algorithms

problem. They are *P*-move and *R*-move. Corresponding to *P*-move and *R*-move, there exist two kinds of search neighborhoods respectively: *P* neighborhood and *R* neighborhood.

• *P*-move

The *P*-move is composed of three steps.

- (1) Randomly choose a course from the current

state.

- (2) Randomly choose a lecture from the course.

- (3) Re-schedule the lecture to another empty period randomly.

Current state is changed to a neighbor state in its *P* neighborhood by a *P*-move. Fig. 2 shows how a *P*-move works.

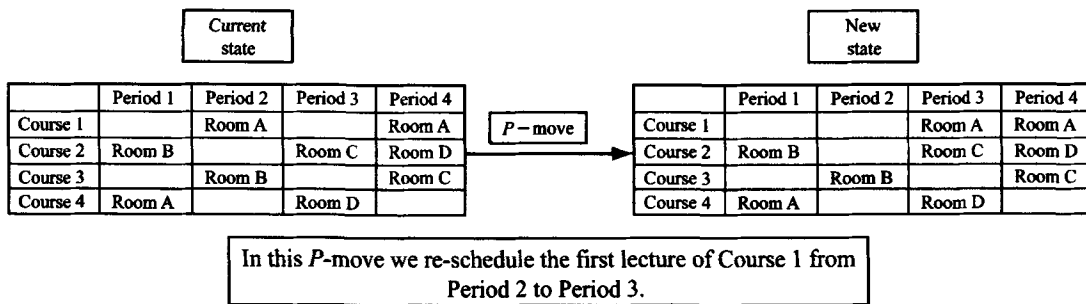


Fig.2 Operation of *P*-move

• *R*-move

The *R*-move is also composed of three steps.

- (1) Randomly choose a course from the current state.

- (2) Randomly choose a lecture from the course.

- (3) Reassign another classroom to the lecture randomly.

Current state is changed to a neighbor state in its *R* neighborhood by a *R*-move. Fig. 3 shows how a *R*-move works.

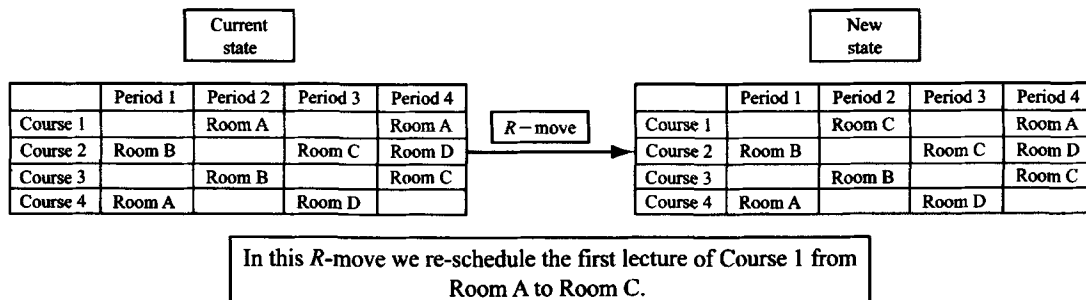


Fig.3 Operation of *R*-move

In using multiple-neighborhood-based parallel composite local search algorithm to solve coursetable problem, we use two kinds of search neighborhoods: *P* and *R*, three kinds of local search algorithm: hill climbing, simulated annealing, and Tabu search. We used $HC(P)$ to denote hill climbing combined with search neighborhood *P*, and $SA(R)$ to denote simulated annealing combined with search neighborhood *R*, respectively.

Fig. 4 shows how multiple-neighborhood-based parallel composite local search algorithm is applied in the coursetable problem. Note that we use $HC(P)$ and $HC(R)$ to improve the initial solution^[13,14].

4 Experimental Results

Implementation of the proposed algorithm for the coursetable problem is based on MPICH-1.2.2.1, and EasyLocal++ 2.0.MPI (message-passing interface) is a standard specification for message-passing libraries. MPICH is a freely available implementation of the MPI standard that runs on a wide variety of systems. EasyLocal++ is an object-oriented framework that helps the user to design and implement local search algorithms in C++ for a large variety of problems. The implementation ran on the clustering computers system of Shanghai University, ZiQiang 2000.

The experimental data are listed in Table 1.

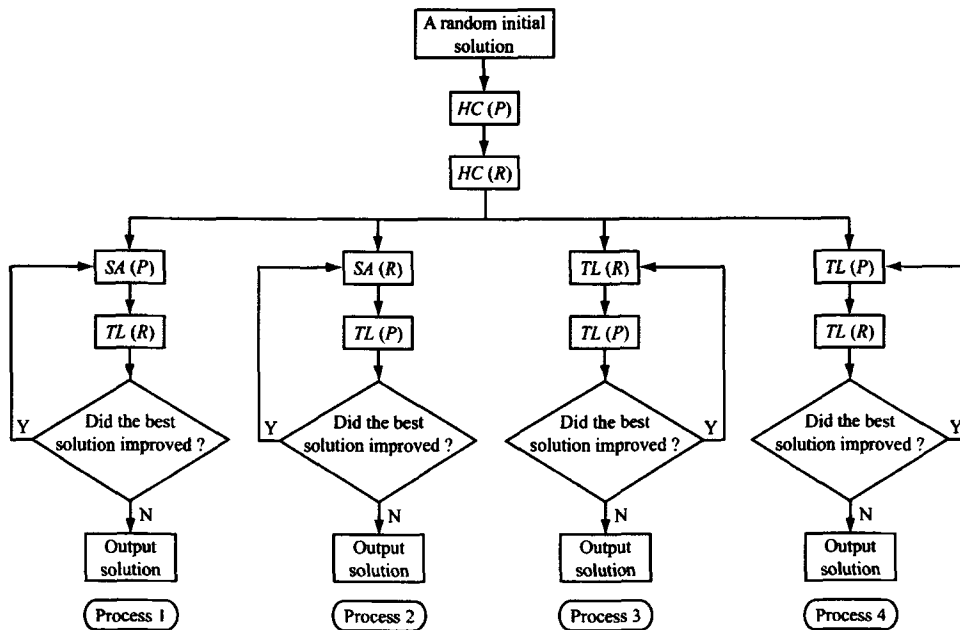


Fig. 4 Multiple-neighborhood-based parallel composite local search algorithm applied in the coursetable problem

Table 1 Experimental data

The number of courses	The number of teachers	The number of periods	The number of rooms
30	28	16	17

Now we give the experimental results of three general local search algorithms.

(1) Hill climbing (combined with search neighborhood R)

Some arguments are listed as follows:

- Max - Noimprove - Iterations: 300

The results of $HC(R)$ run five times with random initial solution are listed in Table 2.

Table 2 Results of $HC(R)$ run five times with random initial solution

Run No.	Execute time	Time of break hard constraints	Times of break soft constraints	Value of evaluation function
1	0.03 s	51	37	51037
2	0.02 s	49	33	49033
3	0.03 s	43	29	43029
4	0.03 s	64	35	64035
5	0.02 s	59	27	59027
Average	0.026 s	53.2	32.2	53232

(2) Hill climbing (combined with search neighborhood P)

Some arguments are listed as follows:

- Max - Noimprove - Iterations: 300

The results of $HC(P)$ run five times with random

initial solution are listed in Table 3.

(3) Simulated annealing (combined with search neighborhood R)

Some arguments are listed as follows:

- Initial - Temperature: 20000

Table 3 Results of $HC(P)$ run five times with random initial solution

Run No.	Execute time	Time of break hard constraints	Times of break soft constraints	Value of evaluation function
1	0.08 s	4	54	4054
2	0.09 s	5	56	5056
3	0.071 s	4	60	4060
4	0.10 s	6	57	6057
5	0.09 s	2	59	2059
Average	0.0862 s	4.2	57.2	4257

- Cooling - Rate: 0.99
- neighbors - sampled: 200

The results of $SA(R)$ run five times with random initial solution are listed in Table 4.

(4) Simulated annealing (combined with search neighborhood P)

Some arguments are listed as follows:

- Initial - Temperature: 20000
- Cooling - Rate: 0.99
- neighbors - sampled: 200

The results of $SA(P)$ run five times with random initial solution are listed in Table 5.

Table 4 Results of $SA(R)$ run five times with random initial solution

Run No.	Execute time	Time of break hard constraints	Times of break soft constraints	Value of evaluation function
1	11.296 s	50	33	50033
2	10.635 s	54	21	54021
3	10.616 s	55	25	55025
4	11.596 s	43	22	43022
5	10.946 s	50	31	50031
Average	11.0178 s	50.4	26.4	50426

Table 5 Results of $SA(P)$ run five times with random initial solution

Run No.	Execute time	Time of break hard constraints	Times of break soft constraints	Value of evaluation function
1	17.805 s	0	46	46
2	18.397 s	2	52	2052
3	20.87 s	0	57	57
4	18.927 s	0	56	56
5	18.256 s	0	59	59
Average	18.851 s	0.4	54	454

(5) Tabu search (combined with search neighborhood R)

Some arguments are listed as follows:

- Max_Length of TabuList: 60
- Min_Length of TabuList: 40
- Max_Noimprove_Iterations: 40

The results of $TL(R)$ run five times with random initial solution are listed in Table 6.

Table 6 Results of $TL(R)$ run five times with random initial solution

Run No.	Execute time	Time of break hard constraints	Times of break soft constraints	Value of evaluation function
1	10.685 s	45	27	45027
2	9.764 s	55	27	55027
3	9.644 s	54	25	54025
4	12.889 s	57	26	57026
5	16.403 s	61	26	61026
Average	11.877 s	54.4	26.2	54426

(6) Tabu search (combined with search neighborhood P)

Some arguments are listed as follows:

- Max_Length of TabuList: 60
- Min_Length of TabuList: 40
- Max_Noimprove_Iterations: 40

The results of $TL(P)$ run five times with random initial solution are listed in Table 7.

From the above experimental results, it is found that an algorithm combined with search neighborhood

Table 7 Results of $TL(P)$ run five times with random initial solution

Run No.	Execute time	Time of break hard constraints	Times of break soft constraints	Value of evaluation function
1	12.518 s	0	58	58
2	11.326 s	0	49	49
3	8.322 s	0	55	55
4	16.053 s	1	65	1065
5	11.196 s	0	59	59
Average	11.883 s	0.2	57.2	257.2

P can generate better solution than the algorithm combined with search neighborhood R . It is also found that the execution time of hill climbing is less than other algorithms. The solution's quality of simulated annealing and tabu search is better in quality of the solution than hill climbing.

However, any algorithm (combined with any search neighborhood) cannot generate the optimal solution with the penalty function value equal to zero. This is why we apply parallel composite local search algorithm to the coursetable problem.

Now we give experimental results of the parallel composite local search algorithm on the same experimental data in Table 8. The arguments of these algorithms that compose the parallel composite local search algorithm are the same as the above arguments.

From these results, it is found that based on the experimental data the parallel composite local search algorithm can always generate an optimal solution. But for the different initial solutions, composition of the composite algorithm that can generate the optimal solution is different. The relation between the initial solution and the composition of composite algorithm is still a problem.

5 Conclusions

In this paper we have presented a parallel composite local search algorithm based on multiple search neighborhoods to solve a special kind of timetable problem—coursetable problem. The results shown in this work indicate that the algorithm is quite effective in finding the optimal solution from an enormous search space.

Finally, there are several issues that are worth further investigation:

Table 8 Results of the parallel composite local search algorithm run five times with five different random initial solutions

Run No.	Initial solution	Process No.	Composite algorithm	Execute time	Time of break hard consdtraints	Time of break soft constrains	Final solution
1	90088	1	SA(P) + TL(R)	21.691 s	0	0	0
		2	SA(R) + TL(P)	53.848 s	0	3	3
		3	TL(R) + SA(P)	75.329 s	0	2	2
		4	TL(P) + SA(R)	42.515 s	0	3	3
2	86073	1	SA(P) + TL(R)	23.614 s	0	0	0
		2	SA(R) + TL(P)	71.063 s	0	2	2
		3	TL(R) + SA(P)	36.172 s	0	0	0
		4	TL(P) + SA(R)	45.766 s	0	5	5
3	79080	1	SA(P) + TL(R)	63.215 s	0	2	2
		2	SA(R) + TL(P)	57.836 s	0	3	3
		3	TL(R) + SA(P)	55.199 s	0	0	0
		4	TL(P) + SA(R)	52.385 s	0	0	0
4	92074	1	SA(P) + TL(R)	26.368 s	0	0	0
		2	SA(R) + TL(P)	65.173 s	0	4	4
		3	TL(R) + SA(P)	53.187 s	0	0	0
		4	TL(P) + SA(R)	52.766 s	0	3	3
5	80076	1	SA(P) + TL(R)	93.855 s	0	1	1
		2	SA(R) + TL(P)	41.039 s	0	0	0
		3	TL(R) + SA(P)	43.893 s	0	0	0
		4	TL(P) + SA(R)	62.76 s	0	2	2

(1) Experiments on the larger instances of coursetable problem to test our algorithm further.

(2) The relation between initial solution and the composition of composite algorithm.

(3) Embedding of a single local search algorithm such as SA and TS in multiple search neighborhoods.

References

- [1] Kang Lishan, Xie Yun, You Shiyong, *et al.* Nonnumerical Parallel Algorithm (The First Volume): Simulated Annealing Algorithm[M]. Science Press, Beijing, 1995 (in Chinese).
- [2] Wang Ling. Intelligent Optimization Algorithms with Applications[M]. Tsinghua University Press, Beijing, 2001 (in Chinese).
- [3] Lu Kaicheng. Introduce to Algorithm—Algorithm Design and Analyze[M]. Tsinghua University Press, Beijing, 1996 (in Chinese).
- [4] Edmund K B, Sanja P. Recent research directions in automated timetabling[J]. *European Journal of Operational Research*, 2002, 7:18–27.
- [5] Abramson D A, Abela J. A parallel genetic algorithm for solving the school timetabling problem[A]. *IJCAI workshop on Parallel Processing in AI*[C], Sydney, 1991, 17:70–78.
- [6] Andrea S. Local search techniques for large high school timetabling problems[J]. *IEEE Transactions on systems, man, and cybernetics part a: systems and humans*, 1999, 29(4):368–377.
- [7] Costa D. A tabu search algorithm for computing an operational timetable[J]. *European Journal of Operational Research*, 1994, 76:98–110.
- [8] Burke E K, Newall J P. A multistage evolutionary algorithm for the timetable problem[J]. *IEEE Transactions on evolutionary computation*, 1997, 3(1):63–74.
- [9] Abramson D. Constructing school timetables using simulated annealing: Sequential and parallel algorithms[J]. *Management Science*, 1991, 37(1):98–113.
- [10] Aarts E, Lenstra J K. Local Search in Combinatorial Optimization[M]. John Wiley & Son, Chichester, 1997.
- [11] Hertz A. Tabu search for large scale timetabling problems[J]. *European Journal of Operational Research*, 1992, 54:39–47.
- [12] Schaerf A. A survey of automated timetabling[J]. *Artificial Intelligence Review*, 1999, 13:87–127.
- [13] Di Gaspero L, Schaerf A. EASYLOCAL++: An object-oriented framework for flexible design of local search algorithms[R]. Technical Report UDMI/13/2000/RR, Dipartimento di Matematica e Informatica, Universit' di Udine, 2000.
- [14] Di Gaspero L, Schaerf A. Writing local search algorithms using EASYLOCAL++[A]. In: VoBands and Woodruff D L. *Optimization Software Class Libraries* [C]. Kluwer Academic Publishers Boston, 2001.

(Executive editor CHEN Ai-ping)